



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
по курсу «Разработка параллельных и распределенных
программ»

«Параллельная реализация решения системы линейных
алгебраических уравнений с помощью OpenMP»

Студент группы ИУ9-51Б Григорьева С. А.

Преподаватель Царев А. С.

Москва 2025

1 Задача

1. Написать программу, которая реализует итерационный алгоритм решения системы линейных алгебраических уравнений вида $Ax=b$ в соответствии с выбранным вариантом. Здесь A – матрица размером $N \times N$, x и b – векторы длины N . Тип элементов – `double`.

2. Программу распараллелить с помощью OpenMP с разрезанием матрицы A по строкам на близкие по размеру, возможно не одинаковые, части. Реализовать два варианта программы:

1: векторы x и b дублируются в каждом процессе,

2: векторы x и b разрезаются между процессами аналогично матрице A .

Уделить внимание тому, чтобы при запуске программы на различном числе процессов решалась одна и та же задача (исходные данные заполнялись одинаковым образом).

3. Замерить время работы двух вариантов программы при использовании различного числа потоков: 1, 2, 4, 8, 16. Построить график зависимости времени работы программы, ускорения и эффективности распараллеливания от числа используемых ядер. Исходные данные, параметры N и ε подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд. Также параметр N разрешено подобрать таким образом, чтобы он нацело делился на 1, 2, 4, 8 и 16.

4. На основании полученных результатов сделать вывод о целесообразности использования одного или второго варианта программы.

Метод минимальных невязок

В методе минимальных невязок преобразование решения на каждом шаге задается следующими формулами:

$$\begin{aligned}y^n &= Ax^n - b, \\ \tau^n &= \frac{(y^n, Ay^n)}{(Ay^n, Ay^n)}, \\ x^{n+1} &= x^n - \tau^n y^n,\end{aligned}$$

где $(u, v) = \sum_{i=0}^{N-1} u_i v_i$. В отличие от метода простой итерации, метод минимальных невязок содержит два умножения матрицы на вектор на каждой итерации, однако сходится до нужной точности за меньшее число итераций. Критерий завершения счета можно взять такой же, как в методе простой итерации.

2 Практическая реализация

Исходный код программы представлен в листингах 1– 2– 3.

Листинг 1 — Вариант 1

```

1 #include <cmath>
2 #include <vector>
3 #include <iostream>
4 #include <random>
5 #include <omp.h>
6 constexpr int N = 60000;
7 constexpr double EPS = 1e-5;
8 constexpr int MAX_ITER = 100;
9
10 int main() {
11     std::vector<double> x(N, 0.0);
12     std::vector<double> b(N, 0.0);
13     std::vector<double> y(N, 0.0);
14     std::vector<double> Ay(N, 0.0);
15     std::mt19937 gen(42);
16     std::uniform_real_distribution<double> dist(0.0, 1.0);
17     for (int i = 0; i < N; ++i) b[i] = dist(gen);
18     double b2 = 0.0;
19     #pragma omp parallel for reduction(+:b2)
20     for (int i = 0; i < N; ++i) b2 += b[i] * b[i];
21     double b_norm = std::sqrt(b2);
22     if (b_norm == 0.0) return 0;
23     int iter = 0;
24     double rel_res = 0.0;
25     double t_start = omp_get_wtime();
26     while (iter < MAX_ITER) {
27         #pragma omp parallel for
28         for (int i = 0; i < N; ++i) {
29             double sum = 2.0 * x[i];
30             for (int j = 0; j < N; ++j) if (i != j) sum += x[j];
31             y[i] = sum - b[i];
32         }
33         double r2 = 0.0;
34         #pragma omp parallel for reduction(+:r2)
35         for (int i = 0; i < N; ++i) r2 += y[i] * y[i];
36         rel_res = std::sqrt(r2) / b_norm;
37         if (rel_res < EPS) break;
38         #pragma omp parallel for
39         for (int i = 0; i < N; ++i) {
40             double sum = 2.0 * y[i];
41             for (int j = 0; j < N; ++j) if (i != j) sum += y[j];
42             Ay[i] = sum;
43         }
44         double num = 0.0, den = 0.0;
45         #pragma omp parallel for reduction(+:num, den)
46         for (int i = 0; i < N; ++i) {
47             num += y[i] * Ay[i];
48             den += Ay[i] * Ay[i];
49         }
50         if (den == 0.0) break;
51         double tau = num / den;
52         #pragma omp parallel for
53         for (int i = 0; i < N; ++i) x[i] -= tau * y[i];
54         ++iter;
55     }
56     double t_end = omp_get_wtime();
57     std::cout << "Time: " << (t_end - t_start);
58     return 0;
59 }

```

Листинг 2 — Вариант 2 - часть 1

```
1 #include <cmath>
2 #include <vector>
3 #include <iostream>
4 #include <random>
5 #include <omp.h>
6
7 constexpr int N = 60000;
8 constexpr double EPS = 1e-5;
9 constexpr int MAX_ITER = 100;
10
11 int main() {
12     std::vector<double> x(N, 0.0);
13     std::vector<double> b(N);
14     std::vector<double> y(N);
15     std::vector<double> Ay(N);
16     std::mt19937 gen(42);
17     std::uniform_real_distribution<double> dist(0.0, 1.0);
18
19     for (int i = 0; i < N; ++i) {
20         b[i] = dist(gen);
21     }
22
23     double b2 = 0.0;
24     #pragma omp parallel for reduction(+:b2)
25     for (int i = 0; i < N; ++i) {
26         b2 += b[i] * b[i];
27     }
28
29     double b_norm = std::sqrt(b2);
30     int iter = 0;
31     double rel_res = 0.0;
32     double t_start = omp_get_wtime();
```

Листинг 3 — Вариант 2 - часть 2

```

1  while (iter < MAX_ITER) {
2      #pragma omp parallel for
3      for (int i = 0; i < N; ++i) {
4          double s = 2.0 * x[i];
5          for (int j = 0; j < N; ++j) {
6              if (i != j) s += x[j];
7          }
8          y[i] = s - b[i];
9      }
10
11     double r2 = 0.0;
12     #pragma omp parallel for reduction(+:r2)
13     for (int i = 0; i < N; ++i) {
14         r2 += y[i] * y[i];
15     }
16
17     rel_res = std::sqrt(r2) / b_norm;
18     if (rel_res < EPS) break;
19     #pragma omp parallel for
20     for (int i = 0; i < N; ++i) {
21         double s = 2.0 * y[i];
22         for (int j = 0; j < N; ++j) {
23             if (i != j) s += y[j];
24         }
25         Ay[i] = s;
26     }
27
28     double num = 0.0, den = 0.0;
29     #pragma omp parallel for reduction(+:num, den)
30     for (int i = 0; i < N; ++i) {
31         num += y[i] * Ay[i];
32         den += Ay[i] * Ay[i];
33     }
34
35     double tau = num / den;
36     #pragma omp parallel for
37     for (int i = 0; i < N; ++i) {
38         x[i] -= tau * y[i];
39     }
40
41     iter++;
42 }
43 double t_end = omp_get_wtime();
44 std::cout << "Time: " << std::fixed << (t_end - t_start);
45 return 0;
46 }
```

3 Характеристики устройства

Модель оборудования: MEGABOOK T14TA (TECNO T14TA)

Процессор: 11th Gen Intel(R) Core(TM) i5-115

Память: 16 ГиБ

Емкость диска: 512 ГБ

4 Время работы программ

1 вариант

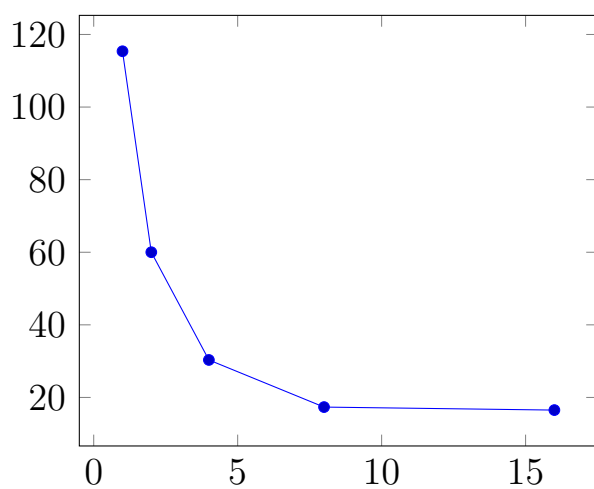
Время вычисления при использовании 1 ядра: 115.373 s

Время вычисления при использовании 2 ядер: 60.0097 s

Время вычисления при использовании 4 ядер: 30.327 s

Время вычисления при использовании 8 ядер: 17.343 s

Время вычисления при использовании 16 ядер: 16.3223 s



2 вариант

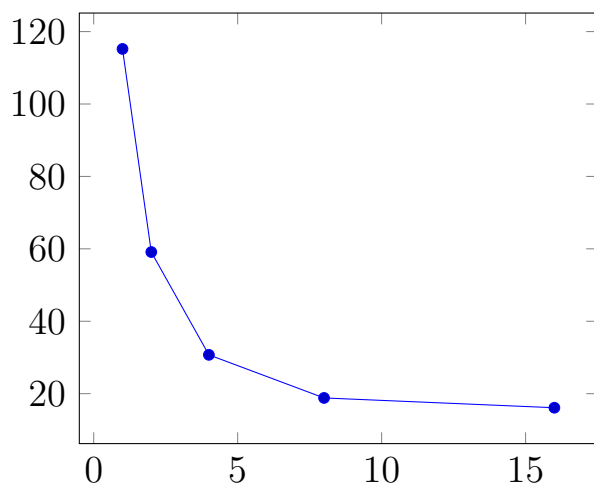
Время вычисления при использовании 1 ядра: 115.211142 s

Время вычисления при использовании 2 ядер: 59.116180 s

Время вычисления при использовании 4 ядер: 30.723965 s

Время вычисления при использовании 8 ядер: 18.821578 s

Время вычисления при использовании 16 ядер: 16.118366 s



5 Вывод

В ходе выполнения лабораторной работы были разработаны две программы для решения системы линейных алгебраических уравнений методом минимальных невязок с использованием технологии OpenMP. В результате было получено, что вариант с распределенными векторами показывает примерно такую же производительность, как и вариант с дублированными векторами. Также оба варианта программы демонстрируют существенное ускорение при увеличении числа потоков.